GeoGuessers: What We Did, Why, and What It Means

Internal report for the team (not for submission)

Swayam Goyal Siddharth Choudhary Yakkala Viswanadha Kartikeya

Contents

1 The project in one page **2**

2 Concepts you need (glossary) **2**

2.1 The terrain operators and their formulas . . . 3

3 What we did, in order **4**

4 The data **5**

5 How the measurements work **6**

5.1 One run . . . 6

5.2 Cold cache . . . 6

5.3 Protocol . . . 6

5.4 Why we pin to performance cores . . . 6

5.5 Why only two folders are excluded from Defender . . . 6

5.6 Splitting a run into stages . . . 6

5.7 Experiments . . . 7

6 Results, and what each one means **7**

6.1 E1: how gdaldem scales . . . 7

6.2 E2: does the file layout matter? . . . 8

6.3 QGIS: why the pyramid matters . . . 9

6.4 Is the data trustworthy? . . . 9

7 Mistakes we made and what we learned **10**

8 Future scope, and how we will do it **10**

8.1 Phase 3: our own engine (E3 CPU, E4 GPU) . . . 10

8.2 Phase 4: computing from the pyramid, and how wrong it is (E5, gap G2) . . . 11

8.3 Phase 5: interactive viewer (E6) . . . 11

8.4 Phase 6 (stretch): viewshed and terrain distance . . . 11

8.5 Phase 7: report and demo . . . 11

8.6 Schedule and owners . . . 12

8.7 Questions to ask the professor . . . 12

9 How to reproduce everything **12**

1 The project in one page

The question. Our professor asked us to work like a systems team: take a very large, high-resolution DEM (Himalaya, 30 m down to 1 m), use only a laptop, and find out (a) where today's standard tools (GDAL, QGIS) stop being practical and *why* (what is the real bottleneck: disk, RAM, CPU, GPU transfer, rendering?), and (b) how to process and display only what the user is looking at, using a **pyramid** (the same DEM stored at several resolutions), computing slope/aspect/hillshade **on demand** and using the CPU and GPU where each actually helps.

What is finished. Phase 0 (software setup), Phase 1 (data) and Phase 2 (baseline benchmarks of the standard tools). Phase 2 answers part (a). Parts of (b) start in Phase 3.

The answer so far, in three lines.

1. `gdaldem` (GDAL's terrain tool, which QGIS also calls) **never runs out of memory**: it handles 10.8 billion cells (43 GB) with 0.83 GB of RAM. It just gets slower: 3.5 min for one slope pass at 1 m.
2. Its bottleneck is **computation on a single CPU thread** (73% of the time; it uses 1 of our 22 threads). The disk and RAM are nowhere near their limits.
3. For **display**, the bottleneck is **reading full-resolution data**: without a pyramid QGIS needs 32 s to draw the whole 1 m area; with a pyramid, 0.09 s at any size.

So the plan for the rest of the project is clear: build our own engine that uses all cores (and the GPU), reads and decompresses in parallel, computes several products in one pass, and works from the pyramid on demand.

2 Concepts you need (glossary)

Term	Meaning, and why it matters for us
DEM	Digital Elevation Model: a grid (raster) where each cell stores a ground height.
Resolution vs cells	Halving the cell size quadruples the number of cells. Our area at 8 m is 169 M cells; at 2 m 2.7 B; at 1 m 10.8 B. Run time follows the number of cells, which is why we talk in cells, not metres.
float32	4 bytes per cell. 10.8 B cells × 4 B = 43 GB, i.e. 2.7× our 16 GB RAM.
CRS, UTM 44N	Coordinate reference system. Slope needs the cell size in metres, so everything is reprojected to UTM zone 44N (EPSG:32644), a metric system for our longitude. Degrees would give wrong slopes.
Reprojection / resampling	Moving a grid onto a new grid. *Nearest* copies the closest value (keeps values, blocky), *bilinear* and *cubic* blend neighbours (smooth), *average* averages all cells that fall into a coarser cell (used to build pyramids).
GeoTIFF, strips, tiles	The file format. Data can be stored in rows (strips) or squares (tiles, e.g. 512×512). Tiles let a program read a small window without reading whole rows.
Compression	LZW, DEFLATE, ZSTD shrink the file; the price is CPU time to decompress on every read. The *predictor* (floating-point predictor = 3) makes height data compress much better.

Term	Meaning, and why it matters for us
COG	Cloud-Optimized GeoTIFF: a tiled GeoTIFF with its pyramid stored inside, laid out so windows and levels can be read efficiently. All our prepared grids are COGs (512 tiles, ZSTD).
Overview / pyramid level	A coarser copy stored inside the file. Level k has cell size $2^k\times$ base (8 m $\rightarrow$ 16, 32, 64 … m). Displaying a zoomed-out view from a coarse level reads far less data.
3×3 window operators	Slope, aspect, hillshade, TRI, TPI and roughness each look at a cell and its 8 neighbours. Cost: one visit per cell, so O(N). Faster versions differ in *implementation*, not in the maths.
Horn vs ZT	Two ways to estimate the height gradient from the window (formulas in Sec. 2.1). Horn uses all 8 neighbours with weights; Zevenbergen–Thorne (ZT) only the 4 direct ones. GDAL's default is Horn.
Halo	When a big grid is cut into tiles, each tile needs one extra ring of cells from its neighbours so 3×3 operators are exact at tile edges. Needed for our engine.
File cache (page cache)	Windows keeps recently read file data in spare RAM (the *standby list*). A second read can then come from RAM ("warm") instead of disk ("cold").
Cold / warm run	Cold: we empty the cache first, so the file must come from disk. Warm: immediate repeat.
Peak working set	The most RAM a process held at any moment; our "peak memory".
ns per cell	Run time divided by the number of cells, in nanoseconds. Makes grids of different size comparable.
Median, spread	We repeat each run 3–5 times and report the median (robust to one odd run). Spread = (max − min) / median.
P-, E-, LP-cores	Our Intel CPU has 6 fast performance cores (12 threads), 8 slower efficiency cores and 2 very slow low-power cores. Where Windows puts a process changes its speed by up to 2.7×.
Turbo budget	The CPU shares a power/heat budget across cores: if other cores are busy, the active core runs at a lower clock. This is how a background process can slow us down without using "our" core.
Defender scan, `$Deleted`	Windows Defender scans new files. If we delete a file it is still scanning, Windows keeps its space reserved in a hidden NTFS area (`$Deleted`) until the scan ends.
Vertical datum	What height zero means. HMA 8 m and Chamoli 2 m use the WGS84 ellipsoid; Copernicus uses the EGM2008 geoid. Here they differ by about 24.5 m. A constant offset does not change slope, but matters for viewsheds.
NMAD	Normalised median absolute deviation: a robust version of standard deviation used to compare DEMs.

2.1 The terrain operators and their formulas

Label the 3×3 window around cell e as

$$\begin{array}{ccc} a & b & c \\ d & e & f \\ g & h & i \end{array} \qquad \text{cell size } \Delta x = \Delta y.$$

Horn (1981):

$$\frac{\partial z}{\partial x} = \frac{(c + 2f + i) - (a + 2d + g)}{8\Delta x}, \qquad \frac{\partial z}{\partial y} = \frac{(g + 2h + i) - (a + 2b + c)}{8\Delta y}.$$

Zevenbergen–Thorne (1987):

$$\frac{\partial z}{\partial x} = \frac{f-d}{2\Delta x}, \qquad \frac{\partial z}{\partial y} = \frac{h-b}{2\Delta y}.$$

Then **slope** $= \arctan\sqrt{(\partial z/\partial x)^2 + (\partial z/\partial y)^2}$; **aspect** is the direction of the steepest descent, from atan2 of the two gradients; **hillshade** (light from azimuth ϕ, altitude θ) $= 255\,[\sin\theta\cos s + \cos\theta\sin s\cos(\phi - \text{aspect})]$; **TRI** (Riley) $= \sqrt{\sum(z_n - z_e)^2}$ and (Wilson) the mean of $|z_n - z_e|$ over the 8 neighbours n; **TPI** $= z_e - \text{mean}(z_n)$; **roughness** $=$ max $-$ min of the window.

Two consequences used later: (1) every operator divides by the cell size, so slope computed on a coarser pyramid level is *different* (smoother), not just faster; that is exactly what experiment E5 will measure. (2) All six need the same few neighbour differences, so computing them together (one pass) shares most of the work.

3 What we did, in order

1. **Setup.** Python environment `geo` (GDAL 3.13.3, rasterio, Numba, CuPy, FastAPI) via Miniforge at `D:\miniforge3` (the default path failed because our user folder name has a space), QGIS 3.44.14. Checked: CuPy runs on the RTX 4060; QGIS slope matched an OpenTopography reference to 10^{-5} degrees.
2. **Choosing data.** The first downloads (an Everest 90 m crop, two single 8 m strips) were too small or patchy. We switched to the NASA High Mountain Asia 8 m *mosaic* tile-571 (Uttarakhand), downloaded Copernicus 30 m and 90 m for exactly the same box through the OpenTopography API, and found a free real 2 m DEM of the Chamoli valley inside the tile (Bhushan & Shean, Zenodo).
3. **One area, many resolutions.** Run time depends on the number of cells, not on which mountain it is. Using one area means grid size is the only thing that changes between measurements, and the real 2 m data gives us a "truth" for accuracy later.
4. **Checking the data.** Tile-571 is 90.4% valid (gaps in the south-east corner). The 8 m tile is 24.5 m below Copernicus (datum difference, see glossary). The Chamoli 2 m averaged to 8 m matches the 8 m DEM to a median of -0.7 m (NMAD 5.1 m): the two are consistent. Chamoli was re-gridded with nearest-neighbour (only 0.09/0.16 m off our grid) so its heights are untouched.
5. **Are synthetic grids OK?** No free 1 m DEM exists here. We tested on a 16 M-cell crop: an 8 m grid rebuilt from 16 m takes the same gdaldem time as the real one, but its slopes are off by 2.3–3.0° RMSE, and nearest-neighbour data compresses 26% better (fake fast reads). Decision: synthetic 2 m and 1 m grids, made with *cubic* resampling, are used for *timing only.*
6. **Building the harness** (Sec. 5): exact per-process measurements, cold-cache flushing, resumable sweeps.
7. **First E1 pass** (74 min, 560 runs, 0 failures). Then a critical review found weak spots: the "read" reference was wrong, one run was an unexplained 2× outlier, conditions were not recorded.
8. **Fixing the method.** Core test (Sec. 5.4) → pin to performance cores. New pure read reference. Defender's CPU use measured (tiny during runs). Run conditions saved with every sweep.
9. **Second attempt (E1 + E2 + QGIS)** ran into a full disk: 108 GB of deleted outputs were held in `$Deleted` while Defender scanned them. We stopped, deleted those results, freed space, added a safeguard (wait until deleted space is really released before the next run), and re-ran.

10. **Chamoli anomaly.** Some Chamoli runs were 25–35% slower in streaks, with 1–2 GB of disk reads that were not gdaldem's: background scanning of previous outputs. With the safeguard the re-run was clean, but E2 (which writes 10.8 GB outputs) was then stopped by the safeguard itself (35.6 GB held for >15 min).

11. **Final setup and full clean re-run.** We excluded only the two scratch folders from Defender, restarted the laptop (frees `$Deleted`) and re-ran everything under identical conditions: **832 runs, 0 failures**. A quality check confirmed the data are clean (Sec. 6.4). The disturbed attempts are kept as evidence.

4 The data

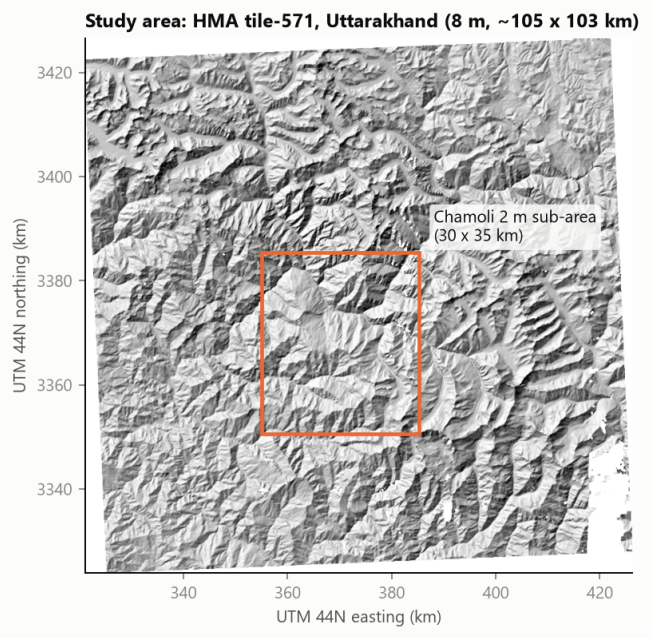


Figure 1: Study area (8 m shaded relief) and the Chamoli 2 m sub-area.

File (`D:\geoguessers_data`)	Res.	Cells	Valid	Notes
`prepared/tile571_90m.tif`	90 m	1.3 M	100%	Copernicus GLO-90
`prepared/tile571_30m.tif`	30 m	12 M	100%	Copernicus GLO-30
`prepared/tile571_8m.tif`	8 m	169 M	84%	HMA mosaic; reprojection leaves thin empty wedges at the tilted edges
`prepared/chamoli_2m.tif`	2 m	264 M	94%	real 2 m; heights identical to the download
`synthetic/tile571_2m_cubic.tif`	2 m	2.7 B	~89%	timing only, 5.0 GB on disk
`synthetic/tile571_1m_cubic.tif`	1 m	10.8 B	~89%	timing only, 16.6 GB on disk

All grids share one origin on a 360 m lattice (divisible by 90, 30, 8, 2 and 1), so they line up exactly. The raw downloads are in `datasets/`; everything is listed with checksums in `datasets/manifest.csv` and is rebuilt by `python datasets/prepare.py [--synthetic]`.

5 How the measurements work

5.1 One run

`bench/harness.py` starts the tool (e.g. `gdaldem slope in.tif out.tif`) as its own process, pinned to the performance cores, and waits. When it ends, the harness reads from Windows, for that process only: wall time, peak memory, total CPU time (so CPU time / wall time = how many cores it used on average), bytes it read and wrote. It also counts bytes physically read from the disks during the run. If the disk read much more than the process did, someone else was using the disk: that is our "foreign disk read" interference signal. One row per run goes into `results/runs.csv`.

An early version sampled these counters every 50 ms and missed the end of short runs; reading the exact totals from the finished process fixed that.

5.2 Cold cache

To measure a true first read we must empty the Windows file cache. That needs admin rights, so we registered a scheduled task (`\GeoGuessers\GeoGuessers-FlushCache`) that runs Sysinternals `RAMMap -Et` (empty standby list) with admin rights; the benchmark triggers it without being admin. We verified it: after a flush the 422 MB 8 m file was read entirely from disk again (at ∼2 GB/s). (An early mistake used `-Ec`, which is not a RAMMap option; RAMMap then just waited.)

5.3 Protocol

Real grids: 5 cold runs (cache emptied before each), then 1 warm-up and 5 warm runs. Synthetic 2 m/1 m: 3 cold + 1 warm (the last cold run doubles as warm-up). This trimmed protocol saved ∼5 hours; operator comparisons do not depend on size, so they come from the real grids. Medians are reported.

5.4 Why we pin to performance cores

We ran the same 8 m slope job pinned to each of the 22 logical CPUs. Performance cores: 3.7–4.1 s. Efficiency cores: ∼6.6 s (1.7×). Low-power cores: up to 10.5 s (2.7×). Unpinned, Windows may move a hidden background process onto slow cores mid-run: that explains one first-pass run of 108.9 s instead of ∼56 s. Pinning makes every tool run at its best, so any speedup we claim later for our engine is *conservative*.

5.5 Why only two folders are excluded from Defender

Defender's CPU use during a run is tiny (0.02–0.05 of a core). The problem is *after* the run: it scans each new 10–40 GB output, and if we delete the file meanwhile, its space stays reserved until the scan ends. That filled our disk once (108 GB) and later slowed the next runs (scan competing for disk and turbo budget). The files in `bench_out` and `e2` are written by our own GDAL tools and deleted within minutes, so excluding just those two folders loses no protection. The harness additionally waits until deleted space is released before starting the next run, and stops cleanly (instead of filling the disk) if that takes more than 15 minutes.

5.6 Splitting a run into stages

gdaldem cannot tell us its internal timings, so we use two reference runs per grid:

- **read**: a small script that reads and decompresses every full-resolution block, nothing else (`bench/read_blocks.py`); it reports its own loop time, excluding Python start-up;

- **copy**: `gdal_translate` to an uncompressed GeoTIFF: the same read plus a same-size write.

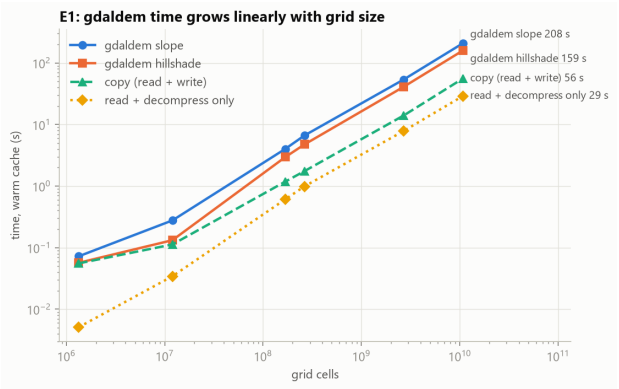
Then *write + tool overhead* ≈ copy − read and *compute* ≈ gdaldem slope − copy. The first read reference (`gdalinfo -checksum`) spent over half its time computing the checksum, which made "write" come out negative; that is why it was replaced.

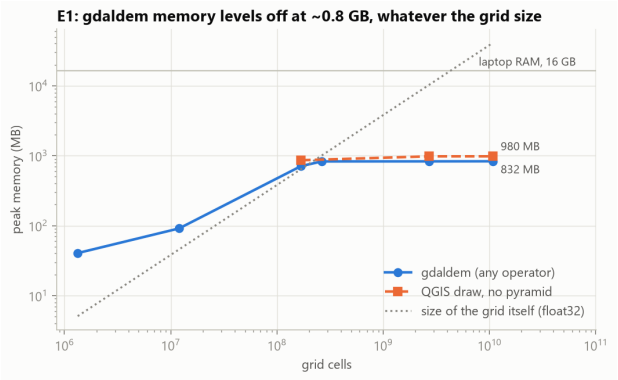
5.7 Experiments

- **E1** (560 runs): 10 gdaldem configurations (slope, aspect, hillshade × Horn/ZT; TRI Riley/Wilson; TPI; roughness) on 6 grids, cold and warm, plus read/copy references. At 1 m only slope and hillshade (Horn).
- **E2** (224 runs): 7 storage layouts of the 8 m and 2 m grids; for each, the time to write it, a pure read, and gdaldem slope. Each 2 m copy (≤10 GB) was moved to the external drive right after its runs.
- **E1-QGIS** (48 runs): QGIS (headless) opens the layer and draws one 1920×1080 image, either the whole area or a 1920 m×1080 m zoom, with the pyramid or with it hidden (GDAL option `OVERVIEW_LEVEL=NONE`). GDAL's statistics caches are disabled so every open starts clean.

6 Results, and what each one means

6.1 E1: how gdaldem scales

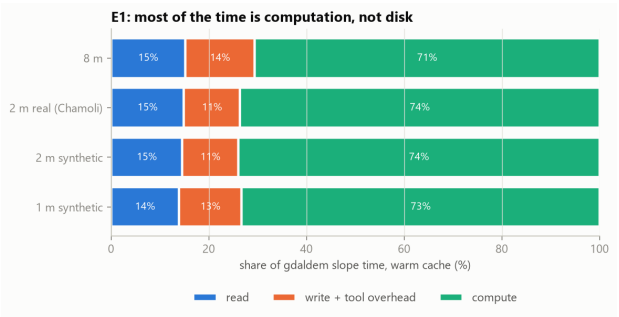


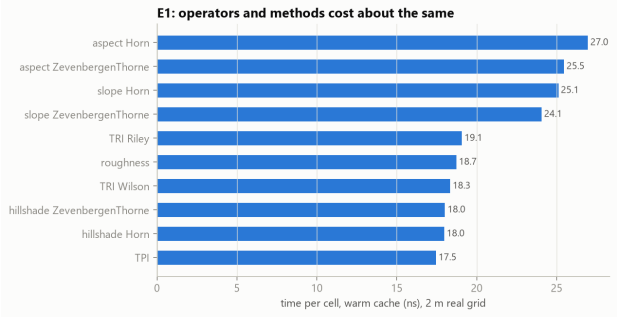


How to read the left chart. Both axes are logarithmic. A straight line at 45° means "time proportional to cells". Slope and hillshade lines are straight from 12 M cells up: **linear scaling**, about 20–25 ns per cell. At the far left (90 m) the line bends because a run lasts only 0.07 s and most of that is starting the program. Numbers: slope 4.0 s (8 m), 6.6 s (2 m real), 54 s (2 m synthetic), 208 s (1 m).

How to read the right chart. The dotted line is the size of the grid itself. gdaldem's memory (blue) *stops growing* at ~830 MB while the grid keeps growing past the 16 GB line. Why: gdaldem reads a few rows, computes, writes, and moves on (streaming), keeping only GDAL's block cache, whose default size is 5% of RAM (≈ 0.8 GB). **Meaning:** on this laptop gdaldem cannot "break" on memory at any size. Our question "where does it break?" therefore has the answer "it doesn't; it becomes slow", and the memory-hungry approaches (loading whole arrays, which we will test in Phase 3) are where real breaking points are.

Cold vs warm. They differ by 2% or less. Two reasons: the NVMe SSD reads at ~2 GB/s, so reading is a small part of the run anyway; and at 2 m/1 m the file is bigger than what Windows keeps cached, so even "warm" runs read 3.6/13.6 GB from disk. Our pilot (lit review) saw 3×; that was timed through QGIS on another laptop, so its first runs most likely included loading the programs.



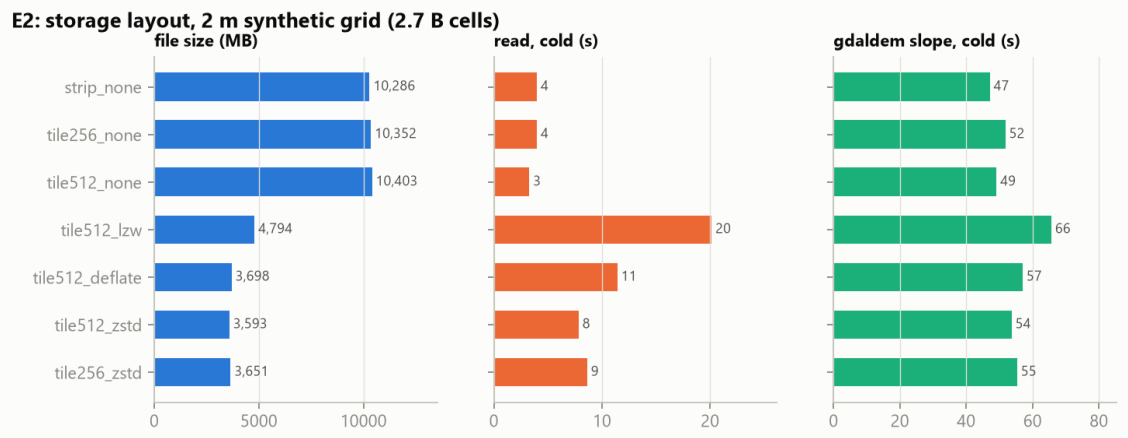


Stage split (left). At every real size, ∼15% is reading + decompressing, 11–14% writing (and tool overhead), and **71–74% computing**. This contradicts the common claim that data movement dominates; for gdaldem on a fast SSD, it is the arithmetic on one thread. **Meaning:** parallelising the computation is the biggest lever, but read and write are ∼27% together, so if only the computation were parallel, the best possible speedup would be about $1/0.27 \approx 3.7\times$ (Amdahl's law). To go further we must also parallelise decompression and writing, or avoid writing at all (on-demand display).

Operators (right). All ten cost 17.5–27 ns per cell. Horn vs ZT differ by at most 6% (ZT is slightly cheaper because it uses 4 neighbours instead of 8). Slope and aspect cost more than hillshade, TRI, TPI and roughness, partly because they write 4-byte floats while hillshade writes 1-byte values. **Meaning:** the formula is not where speed comes from; implementation and data movement are.

Pinning. On the 2 m synthetic grid, unpinned cold runs ranged 55.8–108.9 s (74% spread); pinned, 53.7–55.2 s (2.8%). Proof that core placement, not the tool, caused the outlier.

6.2 E2: does the file layout matter?

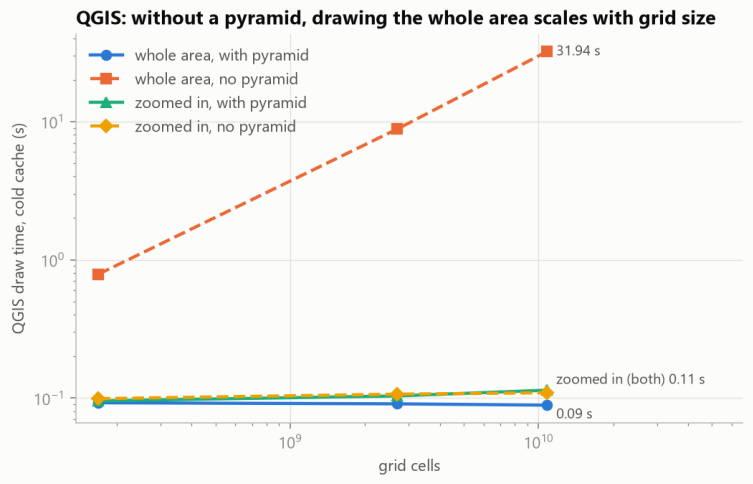


- **Size:** uncompressed 10.3–10.4 GB; LZW 4.8 GB; DEFLATE 3.7 GB; ZSTD 3.6 GB (2.9× smaller).
- **Read:** uncompressed 3.3–4.0 s (pure disk at ∼2.5 GB/s); ZSTD 7.8 s; DEFLATE 11.4 s; LZW 20.1 s. Decompression costs CPU, and LZW decodes slowest.
- **gdaldem slope:** uncompressed strips 47 s (fastest), ZSTD 54 s, DEFLATE 57 s, LZW 66 s (+39%). gdaldem decompresses inside its single thread, so slower decoding directly adds to its time.
- **Write:** compressed files take ∼9× longer to create (ZSTD 124 s vs 14 s). That is a one-time cost when the pyramid is built.

- **Tiles vs strips:** for gdaldem's row-by-row pass, strips are fine (even slightly faster). Tiles matter for reading small windows, which is what the on-demand viewer does; 512 is slightly better than 256.

Meaning: layout changes gdaldem time by up to ∼40%, less than computation but not negligible. Tiled ZSTD, which we already use for the prepared grids, is the right compressed format: smallest and fastest to decode. Avoid LZW. In our parallel engine, decompression can be spread across cores, which should shrink this cost.

6.3 QGIS: why the pyramid matters



- **Whole area, with pyramid:** 0.09 s at 8 m, 2 m and 1 m, peak 200 MB. QGIS picks the pyramid level whose cells match the screen pixels and reads only that small level.
- **Whole area, no pyramid:** 0.79 s, 8.8 s, **31.9 s**; peak ∼980 MB. To draw 1920 pixels across 105,120 columns it must still decompress every tile of the full grid; 31.9 s is close to the 29 s a pure read of the 1 m file takes.
- **Zoomed in:** ∼0.1 s with or without pyramid, because only the window on screen is read.
- **Opening a layer:** 0.02–0.03 s always (QGIS does not read the data when opening).

Meaning: the display bottleneck is "how much full-resolution data must be read for this view". A pyramid makes zoomed-out views cost the same at any grid size, which is exactly the meeting's proposed design. Since slope for a 256×256 tile costs only ∼1.6 ms (65,536 cells × 25 ns), computing terrain products on the fly per tile is cheap; reading the right window at the right level is what matters.

6.4 Is the data trustworthy?

`python -m bench.analysis.quality_check` computes, per grid, the spread between repeats and the number of runs with foreign disk reads. Final run: median spread 0.2–3.3% for every grid ≥169 M cells (Chamoli improved from 7.8% in the disturbed attempt to 2.5%). Remaining outliers are single slow runs (one 8 m aspect run with background disk activity; one Chamoli TRI run) or the very first cold start of Python/QGIS, which only affects the total time, not the tool's own time we report. Medians absorb these. Runs under ∼10 M cells measure mostly program start-up and are not used for per-cell claims.

7 Mistakes we made and what we learned

What went wrong	Lesson / fix
`gdalinfo -checksum` used as "read"	It does extra CPU work; "write" came out negative. A reference must do *only* the thing it measures: `bench/read_blocks.py`.
Counters sampled every 50 ms	Missed the end of short runs; read exact totals from the finished process.
`RAMMap -Ec`	Not a real option; always check flags before automating. Correct: `-Et`.
Unpinned runs	Hybrid CPUs need pinning; one run was 2× slow.
Defender + deleted 10–40 GB files	Deleted files are not freed while scanned: 108 GB filled the disk. Fix: exclude only scratch folders; wait for freed space between runs.
Background activity in Chamoli runs	"Foreign disk reads" are a cheap, reliable interference detector.
Moving small files to the external drive	Only >1 GB run outputs belong there; small files stay in the project, old ones in the local `old files/` folder.
Session temp scripts lost at restart	Copy every one-off script whose result we quote into `old files/` immediately.

8 Future scope, and how we will do it

8.1 Phase 3: our own engine (E3 CPU, E4 GPU)

Goal: beat gdaldem on the same machine by attacking the measured bottleneck.

Design (package `demlab/`):

- `io.py`: read the COG window by window (e.g. 2048×2048 cells) with a 1-cell halo, aligned to the 512-cell tiles; reading/decompressing several windows at once in a thread pool (GDAL releases Python's lock while decoding, so threads really run in parallel).
- `kernels/numpy_k.py`: the reference implementation, vectorised with array slicing: the Horn formula is applied to shifted views of the whole window at once. Simple, easy to verify, single-threaded, whole window in memory.
- `kernels/numba_k.py`: the same formulas in a loop compiled with `@njit(parallel=True)`, with `prange` over rows so all cores share the work. Computes slope, aspect and hillshade *together* from one gradient pass (fused kernel).
- `kernels/cuda_k.py`: CuPy element-wise / raw kernels on the GPU; copy window to GPU, compute, copy back.
- `tiling.py`: loops over windows, runs the kernel, writes results (tiled, optionally compressed in parallel) or returns them in memory for the viewer.

Correctness first: on the 8 m and Chamoli grids our outputs must equal gdaldem's within float tolerance (e.g. $\leq 10^{-3}$ degrees), including nodata handling and edges (gdaldem leaves the outer ring empty unless asked). Tests in `tests/`.

What we will measure:

- E3 (CPU): time vs number of threads (1, 2, 4, 8, 12, 16, 22; performance vs all cores), vs window size, separate vs fused operators, with and without writing output. Also a deliberately

naive "load the whole array" NumPy version: it should run out of memory around 2 m/1 m, giving the report a real breaking point next to gdaldem's.

- E4 (GPU): time to copy to the GPU, kernel time and copy-back time measured separately with CUDA events; at which grid size the GPU starts to win; windows sized to fit the 8 GB of GPU memory.

What we expect (hypotheses to test): if only computation is parallelised, at most ∼3.7× (Amdahl, since read + write ≈ 27%). With parallel decompression and fused operators the floor is set by the disk: reading the 16.6 GB 1 m file takes ∼8 s at 2 GB/s, versus 208 s for gdaldem today. Writing a 43 GB uncompressed output would itself take ∼20 s, so for display we will avoid writing at all. For the GPU, copying is cheap (∼10 GB/s), so the question is whether CPU-side reading/decompression can feed it fast enough.

8.2 Phase 4: computing from the pyramid, and how wrong it is (E5, gap G2)

Method: for each pyramid level k (Chamoli 2 m → 4, 8, 16, 32, 64 m; tile 8 m → 16–128 m), compute slope directly from that level using its true cell size $2^k\times$ base. As "truth", compute slope at full resolution and average it onto the same coarse grid. Compare per cell: RMSE, mean bias (coarse slopes are expected to be *lower*, because averaging smooths the terrain), and the share of cells off by more than 5°; split by slope class (0–15, 15–30, 30–45, >45°), since steep Himalayan terrain should suffer most. Record the time per level. Secondary checks: does our 8 m level from Chamoli match the independent HMA 8 m DEM, and our 32 m/64 m levels the real Copernicus 30 m/90 m? **Output:** an error-vs-time-saved curve: the headline research result. (Early hint: mean slope was 28.8° at 90 m, 31.1° at 30 m and 32.5° at 8 m, i.e. coarser grids flatten the terrain.)

8.3 Phase 5: interactive viewer (E6)

Tile server (`server/app.py`, FastAPI): a map client asks for tile $z/x/y$; the server converts it to a ground area and resolution, picks the pyramid level whose cell size matches, reads that window plus a halo, computes slope/aspect/hillshade with our kernel (CPU or GPU), colours it, and returns a PNG. Recently used tiles are kept in an LRU cache. **Clients:** QGIS (as an XYZ tile layer) and a Cesium web globe (our tiles draped over the 3D terrain, with a switch between products). **Measurements:** tile latency (median and 95th percentile) for (a) tiles precomputed in advance, (b) on-the-fly CPU, (c) on-the-fly GPU; with and without cache; during a scripted pan/zoom session. A demo video as backup for the final presentation.

8.4 Phase 6 (stretch): viewshed and terrain distance

Only if time allows. Viewshed: the exact R3 algorithm vs the faster approximate XDraw vs `gdal_viewshed`, measuring time and the share of cells classified differently from R3 (XDraw's accuracy loss in high relief is gap G3). Heights must first be converted to one vertical datum (the 24.5 m offset matters here). Terrain distance: Dijkstra vs A* on a slope-based cost surface: time, nodes explored, same path.

8.5 Phase 7: report and demo

Final report (introduction, literature, method, results E1–E6, the bottleneck story, limitations), slides, live demo with video fallback, code frozen and tagged.

8.6 Schedule and owners

When (relative)	Work	Owner (as in the plan)
Week 2	Phase 3 engine: CPU kernels + tiling (E3)	Swayam
Week 2	Phase 3 GPU kernels (E4)	Kartikeya
Week 2–3	Phase 4 pyramid accuracy (E5)	Siddharth
Week 2–3	Phase 5 tile server / Cesium (E6)	Swayam (server), Kartikeya (clients)
Week 3	Phase 6 stretch, if on track	Siddharth
End of Week 3	Report, slides, demo	all

8.7 Questions to ask the professor

- Is "gdaldem does not break, it becomes slow; the real limits are display and memory-hungry approaches" the kind of bottleneck answer he wants, or should we also benchmark other GIS tools (e.g. WhiteboxTools, GRASS)?
- Should the viewer target QGIS, Cesium, or both for the final demo?
- Is the GPU part expected to show a speedup, or is an honest "when it helps and when it doesn't" enough?

9 How to reproduce everything

```
conda activate geo
python datasets/prepare.py --synthetic   # rebuild all grids
powershell -ExecutionPolicy Bypass -File bench\run_phase2.ps1
python -m bench.analysis.quality_check   # check the runs
python -m bench.analysis.phase2_report   # tables + charts
python -m bench.analysis.study_area_map  # report map
```

Where	What
`docs/PROJECT_PLAN.md`	the plan, methodology (§6), phase status
`docs/PHASE2_RESULTS.md`	Phase 2 results in Markdown
`results/runs.csv`	every measured run
`results/plots/`, `results/phase2_tables.md`	charts and tables
`results/env/`	conditions of each sweep
`bench/`	harness, sweeps, analysis
`old files/` (local only)	earlier scripts, test outputs and logs, with a README
`K:\spatial project\e2`	the large E2 layout copies